

TMCTS retains the same high-level idea as the previous Transformer-guided MCTS scheduler: a Transformer encoder produces a policy prior that guides MCTS. The main architectural change is that the input is no longer a static array of scheduling tasks only. Instead, the radar action space is represented explicitly as a mixed action-token set:

- token 0: SEARCH
- tokens 1..N: TRACK actions for currently active targets

In the earlier static formulation, each token corresponded only to a candidate task in a schedule-construction problem. In TMCTS, the Transformer must jointly rank:

- whether to search
- which target to track next

Architecture is still Transformer policy, but token semantics are now radar-specific.

Transformer

- Pure policy-only Transformer
- No value head
- Learned CLS token for global context
- Input features per token: 8
- $d_{\text{model}} = 128$
- $n_{\text{head}} = 4$
- $n_{\text{layers}} = 2$
- feedforward dim = 512
- dropout = 0.1
- policy head: $128 \rightarrow 256 \rightarrow 128 \rightarrow 1$
- inactive track tokens are masked so they receive zero probability mass
- token 0 (SEARCH) is always active

TMCTS Training / Search Settings

- window length: 200 ms
- training iterations: 200
- episodes per iteration: 3
- windows per episode: 200

- training rollout count: 200
- inference rollout count: 16
- optimizer: AdamW
- batch size: 256
- learning rate: $1e-5$ to $2e-5$
- replay-based policy updates using MCTS-improved root policies

Algorithm 1: TMCTS Training

```

Input:
- Radar environment E
- Transformer policy  $f_{\theta}$ 
- Training iterations  $I = 100$ 
- Episodes per iteration  $M = 3$ 
- Windows per episode  $H = 200$ 
- Training rollout count  $R_{\text{train}} = 200$ 

Initialize Transformer parameters  $\theta$ 
Initialize replay buffer B

for  $i = 1$  to  $I$  do
  for episode = 1 to  $M$  do
    Reset radar environment E
    for window = 1 to  $H$  do
      Observe current radar state  $s_t$ 
      Convert  $s_t$  into the action-token matrix  $X_t$ 
      Run TMCTS using  $f_{\theta}$  with  $R_{\text{train}}$  rollouts
      Extract improved root policy  $\pi_t$  from MCTS
      Execute selected action  $a_t$  in E
      Store  $(X_t, \pi_t)$  in B
    end for
  end for

  Sample mini-batches from B
  Update  $\theta$  using policy-matching loss
end for

Output:
- trained Transformer policy  $f_{\theta}$ 

```

Algorithm 2: TMCTS Inference

Input:

- Current radar state s_t
- Trained Transformer policy f_{θ}
- Inference rollout count $R_{\text{inf}} = 16$

Observe current radar state s_t

Convert s_t into the action-token matrix X_t

Run Transformer to obtain action priors $P_{\theta}(\cdot | s_t)$

Run TMCTS using P_{θ} with R_{inf} rollouts

Select best action a_t from the search tree

Execute a_t in the current radar window

Repeat at the next window

TMCTS is trained in a search-supervised manner: at each radar window, the Transformer provides priors, MCTS refines them through rollout search, and the resulting root policy is used as the supervision target. During inference, the same Transformer-guided TMCTS loop is used online, but with a smaller rollout budget.